D.K.T.E. Society's Textile and Engineering Institute, Ichalkaranji.

(An Empowered Autonomous Institute, Affiliated to Shivaji University, Kolhapur)

Department of Computer Science & Engineering (AIML)

2026-27



Project SRS On

AI Smart Customer Support

Under the Guidance Of

Prof. V.R.Kambale mam

Submitted By:

1. 23UAM133 Swarupanand Popat Bajbalkar
2. 23UAM116 Rutuja Suhas Puranik
3. 23UAM126 Samruddhi Mahesh Shingade
4. 23UAM137 Mrudula Rajendra Waghmode

Prof. V.R. Kambale	Prof. Dr.L.S. Admuthe	Prof. Dr. S.K.Shirgave
[Project Guide]	[I/C Director]	[Head of Department]

D.K.T.E. Society's Textile and Engineering Institute, Ichalkaranji.

(An Autonomous Institute, Affiliated to Shivaji University,Kolhapur)

Department of Computer Science & Engineering (Artificial Intelligence and Machine Learning)

CERTIFICATE

This is to certify that,

1. 23UAM133 Swarupanand Popat Bajbaljkar
2. 23UAM116 Rutuja Suhas Puranik
3. 23UAM126 Samruddhi Mahesh Shingade
4. 23UAM137 Mrudula Rajendra Waghmode

Have successfully completed the SRS and Design work, of the mini project Entitled,

Ai Smart Customer Support

In partial fulfillment for Final Year B.Tech. CSE(AI) academics. This is the record of their work carried out during the Academic year 2026-2027.

Date: Place: Ichalkaranji

Prof. V.R.Kambale	Prof. Dr.L.S. Admuthe	Prof. Dr.S.K.Shirgave
[Project Guide]	[I/C Director]	[Head of Department]

DECLARATION

We undersigned students of Third Year B. Tech (Computer Science and Engineering [Artificial Intelligence and Machine Learning]) declare that the project work report entitled **"AI-Smart Customer Support"** written and submitted by us, under the guidance of Prof. V.R.Kambale mam is our original work. The empirical findings in this report are based on the data collected by us. The matter assimilated in this report is not the reproduction of any readymade report.

Date:

Place: Ichalkaranji.

Sr.	PRN	Name of Student	Signature
1.	23UAM133	Swarupanand Bajbalkar	
2.	23UAM116	Rutuja Puranik	
3.	23UAM126	Samruddhi Shingade	
4.	23UAM137	Mrudula Waghmode	

Index

Sr. No.	Contents	Page No.
1	Introduction	5
1.1	Problem Description	5
1.2	Problem Statement	5
1.3	Objectives	6
1.4	Scope	6
2	Overall Description	7
2.1	Product Perspective	7
2.2	Product Functions	9
3	Specific Requirements	10
3.1	External Interfaces	10
3.2	Functional Requirements	10
3.3	Performance Requirements	11
3.4	Design Constraints	13
3.5	Database Requirements	13
3.6	Software System Attributes	16
4	Software Design Document	17
5	Algorithms for Functional Requirements	28
6	References	31

1. Introduction

1.1 Problem Description

Customer support systems in many organizations still depend heavily on manual processes for handling customer complaints and queries. Customers often experience delays in receiving responses due to large volumes of support requests and limited availability of support agents. Traditional support methods also make it difficult to identify customer emotions, prioritize issues, and classify requests automatically.

In existing systems, users need to wait for human intervention even for simple issues such as login problems, payment failures, or account-related queries. This increases response time and reduces customer satisfaction.

To overcome these limitations, the proposed **AI Smart Customer Support System** introduces an intelligent support mechanism that combines chatbot interaction, machine learning, automated ticket generation, sentiment analysis, and priority prediction to provide faster and smarter customer support.

1.2 Problem Statement

The current customer support process lacks automation, intelligent query handling, and effective ticket management. Manual classification and delayed responses reduce operational efficiency and negatively affect user experience.

Therefore, there is a need to develop an **AI-based customer support system** capable of:

- Providing instant responses through chatbot interaction.
- Detecting customer sentiment automatically.
- Classifying support issues into categories.
- Generating tickets automatically for unresolved problems.
- Assigning ticket priorities intelligently.
- Improving response time and customer satisfaction.

1.3 Objectives

The main objective of the proposed **AI Smart Customer Support System** is to build an intelligent and automated support platform that improves customer service quality.

Specific objectives include:

1. To develop a secure user authentication system.

Users should be able to register and log in securely using JWT authentication.

2. To provide automated customer assistance.

The chatbot should respond instantly to common customer issues.

3. To implement sentiment analysis.

The system should analyze customer messages and identify whether sentiment is Positive, Negative, or Neutral.

4. To classify support requests automatically.

Machine learning techniques should categorize customer issues into Technical, Billing, Account, or General categories.

5. To automate ticket management.

Unresolved issues should automatically generate support tickets.

6. To support administrative management.

Administrators should monitor, update, and manage support tickets efficiently.

7. To maintain structured and unstructured data.

SQL Server and MongoDB should be used for efficient data storage.

1.4 Scope

The proposed system is a web-based application designed to provide intelligent customer support using a chatbot and ticket management system.

The system allows users to interact with a chatbot for instant solutions. If the issue is not resolved, a support ticket is automatically generated. The system also performs AI-based analysis such as sentiment detection and issue classification.

The system will:

- Provide chatbot-based support
- Automatically create and manage tickets
- Perform AI-based analysis (sentiment, intent, priority)
- Provide admin dashboard for monitoring

The system will not:

- Replace human agents completely
- Provide voice-based support

2. Overall Description

2.1 Product Perspective

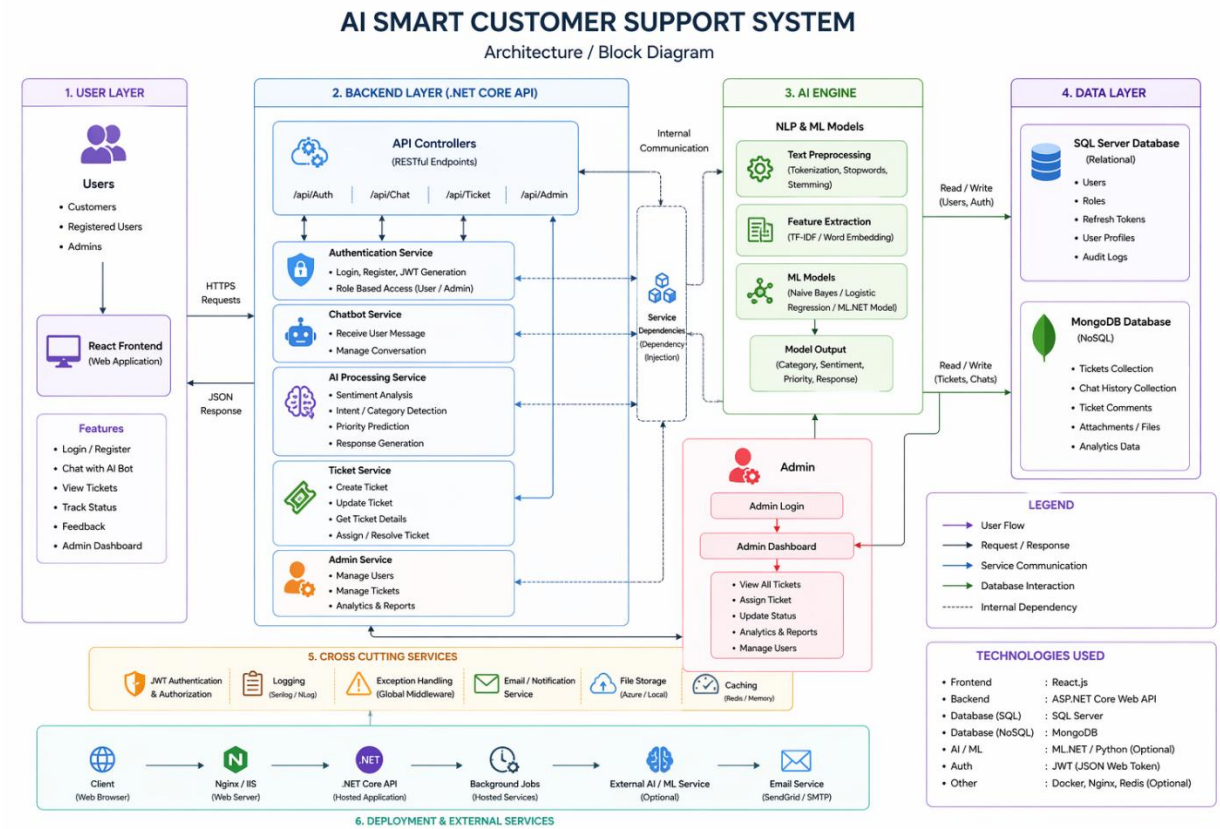
The system is designed as a full-stack web application consisting of frontend, backend, AI module, and databases.

It includes:

- React-based user interface
- .NET Core backend API
- Node.js AI service
- Dual database system (MySQL + MongoDB)

The system follows a modular architecture where each component performs a specific role.

2.1.1 Block Diagram



2.1.2 Hardware Requirement

- Processor: Intel i3/i5 or higher
- RAM: Minimum 8GB
- Storage: 256GB or above
- Internet connection

2.1.3 Software Requirement

Frontend:

- React.js
- HTML, CSS, JavaScript

Backend:

- ASP.NET Core (REST API)

AI Module:

- Node.js (NLP models)

Database:

- **MySQL** → User authentication and admin data
- **MongoDB** → Tickets, chat messages, AI results

Tools:

- Postman
- VS Code / Visual Studio
- GitHub

2.2 Product Functions

The system is designed to make customer support faster and smarter by using AI and automation. It performs several important functions:

- **User Registration and Login:**
  Users can create their account and securely log in to access the system.

- **Chatbot Interaction for Instant Support:**
  Users can ask their questions to the chatbot and get quick responses without waiting for a human agent.

- **Ticket Creation for Unresolved Issues:**
  If the chatbot cannot solve the user's problem, the system automatically creates a support ticket so that it can be handled later.

- **AI-Based Sentiment and Intent Analysis:**
  The system analyzes the user's message to understand their mood (happy, angry, etc.) and what they are asking about.

- **Ticket Categorization and Priority Prediction:**
  Based on the analysis, the system automatically assigns a category (like billing or technical) and sets the priority (low, medium, high).

- **Admin Dashboard for Managing Tickets:**
  Admin can view all tickets, update their status, and monitor system activity through a dashboard.

- **Storage of Data in MySQL and MongoDB:**
  The system stores user-related data in MySQL and stores tickets, chat messages, and AI results in MongoDB for better performance and flexibility.

3.Specific Requirement

3.1 External Interfaces

User Interface:

- Web-based interface for users and admin
- Chat interface for communication

API Interface:

- REST APIs used between frontend and backend

Database Interface:

- MySQL for structured data
- MongoDB for unstructured data

3.2 Functional Requirements

1: User Authentication

The system allow users to register and login securely using authentication mechanisms.

2: Chatbot Interaction

The system allow users to interact with the chatbot and receive instant responses.

3: Ticket Generation

The system create a ticket automatically if the chatbot fails to resolve the query.

4: Ticket Management

The system allow users to view ticket status and allow admin to manage tickets.

5: AI Analysis

The system analyze user messages using AI techniques such as sentiment detection and intent classification.

6: Priority Prediction

The system assign priority levels (Low, Medium, High) based on the severity of the issue.

7: Admin Dashboard

The system provide an admin panel for monitoring, updating, and resolving tickets.

3.3 Performance Required

Static Requirement

Requirement	Description	Expected Value / Target
Concurrent Users	System should handle multiple users at the same time without performance degradation	≥ 100 users simultaneously
Data Storage Capacity	Ability to store large volumes of chat logs, tickets, and user data	Scalable (GB–TB level storage)
Database Performance	Efficient storage and retrieval of structured and unstructured data	Optimized queries with indexing
System Scalability	System should scale horizontally or vertically as user load increases	Cloud-enabled auto scaling
Backup & Recovery	Regular backup of chat and ticket data	Daily automated backups
Data Consistency	Ensure no data loss or corruption during transactions	ACID compliance
Security Performance	Encryption and authentication should not slow down system	Minimal overhead (<5% latency impact)

Dynamic Requirment

Requirement	Description	Expected Value / Target
Chatbot Response Time	Time taken by chatbot to reply to user queries	< 2 seconds
Ticket Processing Time	Time to create, update, or assign a ticket	Within a few seconds (≤ 3 sec)
AI Prediction Time	Time for sentiment analysis, intent detection, and categorization	< 3 seconds
System Throughput	Number of requests handled per second	High throughput (e.g., 100+ req/sec)
API Response Time	Backend API response speed	< 1–2 seconds
Real-Time Updates	Updates in dashboard (tickets, chats) should reflect instantly	Near real-time (< 1 sec delay)
Error Handling Time	Time to detect and respond to system errors	Immediate detection (< 1 sec)
Load Handling	System performance under peak load conditions	No crash up to peak load
Latency	Delay in communication between client and server	Low latency (< 200 ms ideal)
Session Handling	Maintain user sessions efficiently during interaction	Seamless session continuity

3.4 Design Constraints

The proposed system operates under the following design constraints:

Hardware Constraints

- Minimum 8 GB RAM
- Intel i5 processor or higher
- Stable internet connection

Software Constraints

- Windows Operating System
- Visual Studio 2022
- SQL Server Management Studio
- MongoDB Compass
- Python Environment

Technology Constraints

- Backend developed using ASP.NET Core Web API
- Frontend developed using React
- Machine Learning implemented using Python
- SQL Server for authentication data
- MongoDB for ticket and chat storage

Performance Constraints

- System should respond within acceptable time limits.
- Multiple requests should be handled efficiently.

Security Constraints

- JWT Authentication must secure API access.
- User roles should control access permissions.

3.5 Logical Database Requirements

Overview

The system uses a hybrid database architecture by combining both relational and NoSQL databases to achieve better performance, flexibility, and scalability.

- MySQL is used for structured data with strong consistency.
- MongoDB is used for unstructured and dynamic data.

Database Distribution

Database	Data Stored	Purpose
MySQL	Users, Admin	Stores structured data with relationships and ensures data integrity
MongoDB	Tickets, Messages, AI Analysis, Ticket History	Handles flexible, high-volume, and real-time data

Detailed Data Components

1. MySQL (Relational Database)

Used for structured and critical data requiring consistency.

- Users Table
    - User ID, Name, Email, Password
    - Role (Customer/Admin)
    - Authentication details
- Admin Table
    - Admin ID, Name, Role
    - Access permissions
    - Activity logs
- Key Features
    - Structured schema (tables, rows, columns)
    - Supports joins and relationships
    - Ensures ACID properties for data consistency

2. MongoDB (NoSQL Database)

Used for dynamic, large-scale, and semi-structured data.

- Tickets Collection
    - Ticket ID, Issue description
    - Priority, Status, Category
    - Assigned admin
- Messages Collection
    - Chat messages between user and chatbot/admin
    - Timestamp, sender info
- AI Analysis Collection
    - Sentiment analysis results
    - Intent detection
    - Confidence scores
- Ticket History Collection
    - Previous updates and changes in tickets
    - Status tracking logs
- Key Features
    - Schema-less (flexible structure)
    - High scalability and performance
    - Suitable for real-time applications

3.6 Software System Attributes

The software system should satisfy the following quality attributes:

1. Reliability

The system should consistently process customer requests and maintain service availability.

2. Availability

The application should remain accessible to users whenever support services are required.

3. Security

JWT authentication and role-based authorization should protect sensitive information.

4. Scalability

The architecture should support future integration of larger datasets and additional AI models.

5. Performance

Customer queries should be processed efficiently with minimal delay.

6. Maintainability

The modular structure should simplify future updates and enhancements.

7. Usability

The user interface should be easy to understand and operate.

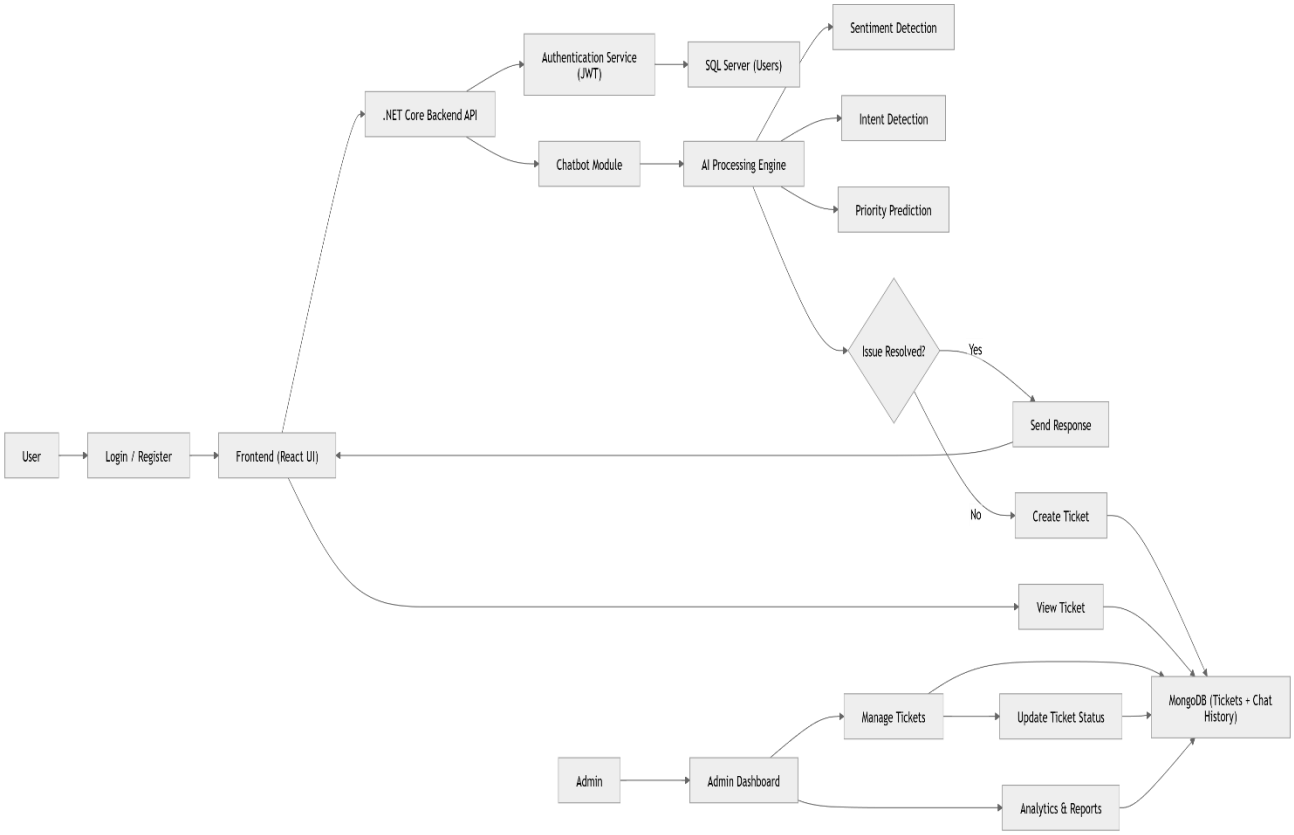
8. Portability

The application should run across different environments with minimal configuration.

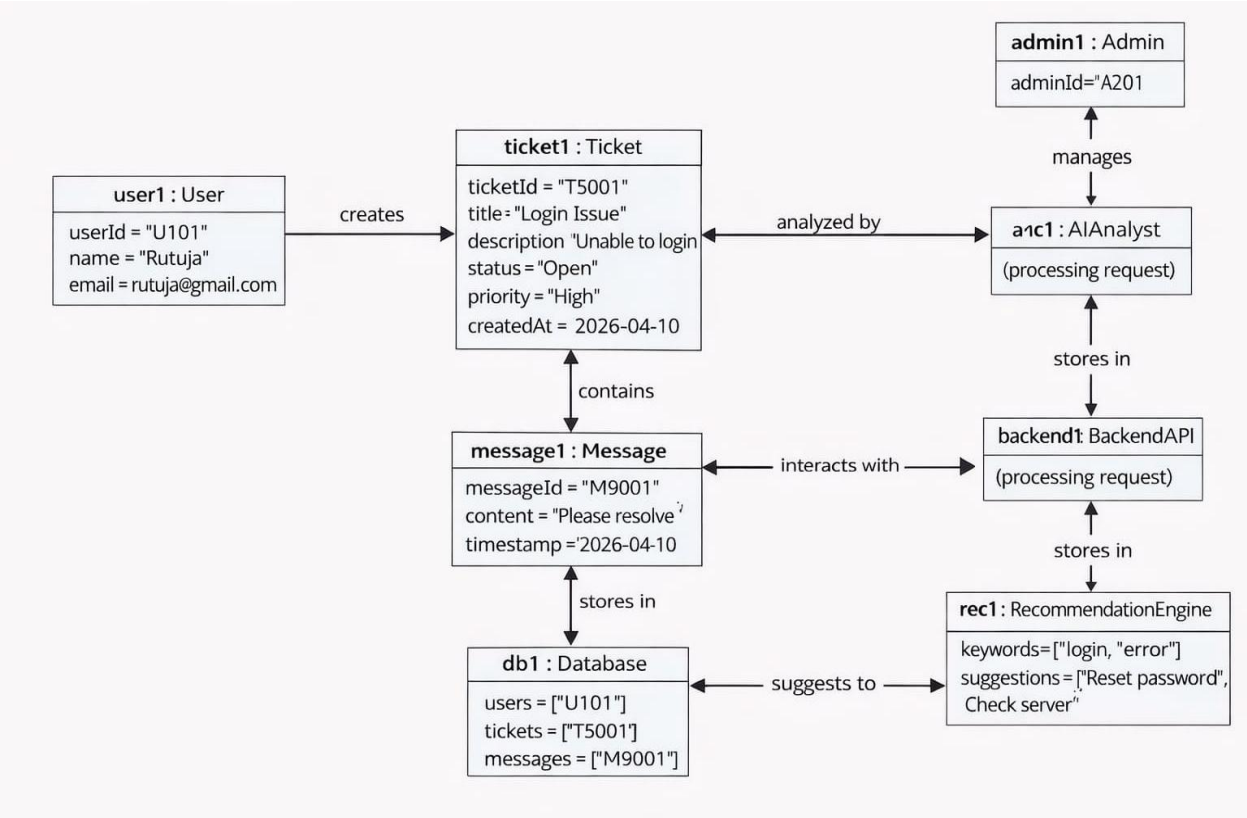
4.Software Design Document

4.1Structural Design

A. Block Diagram

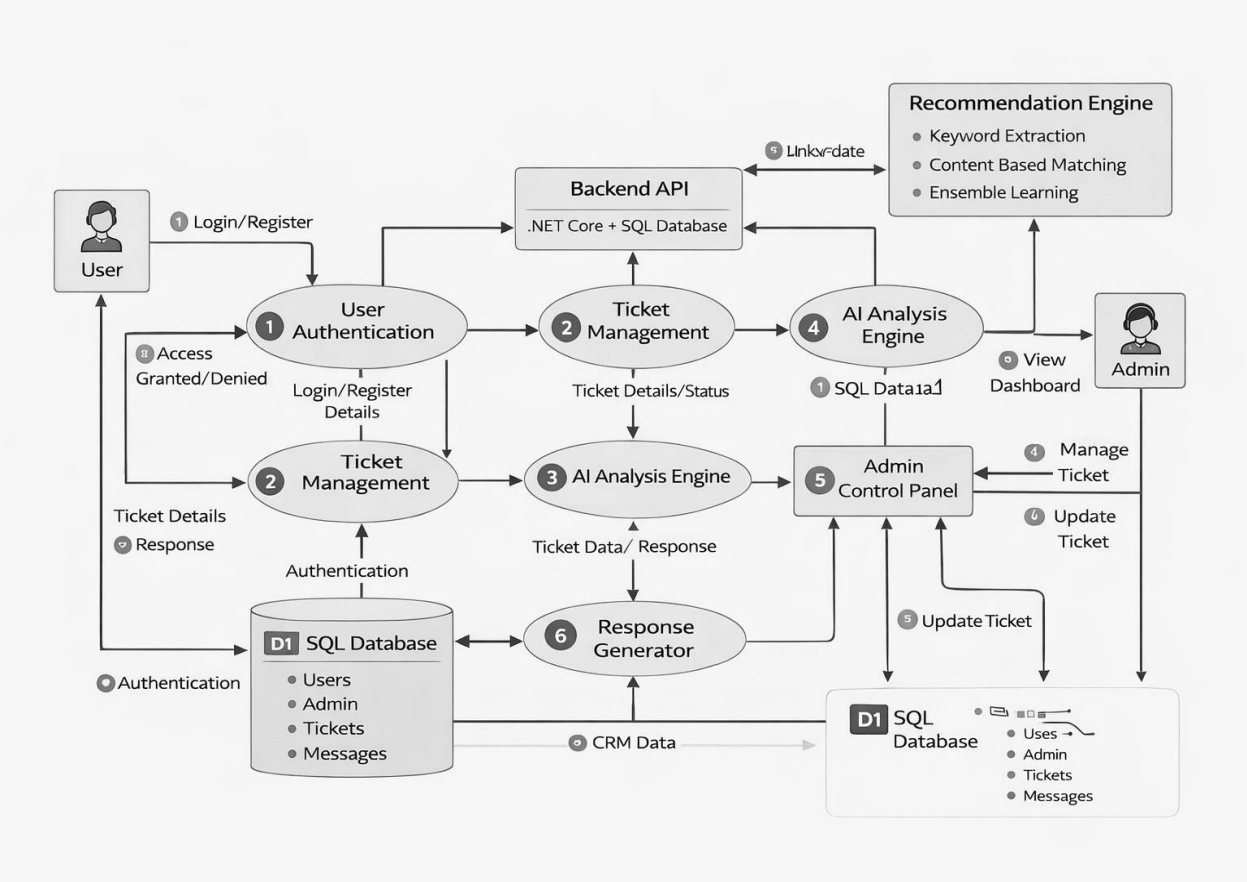


B. Object Diagram

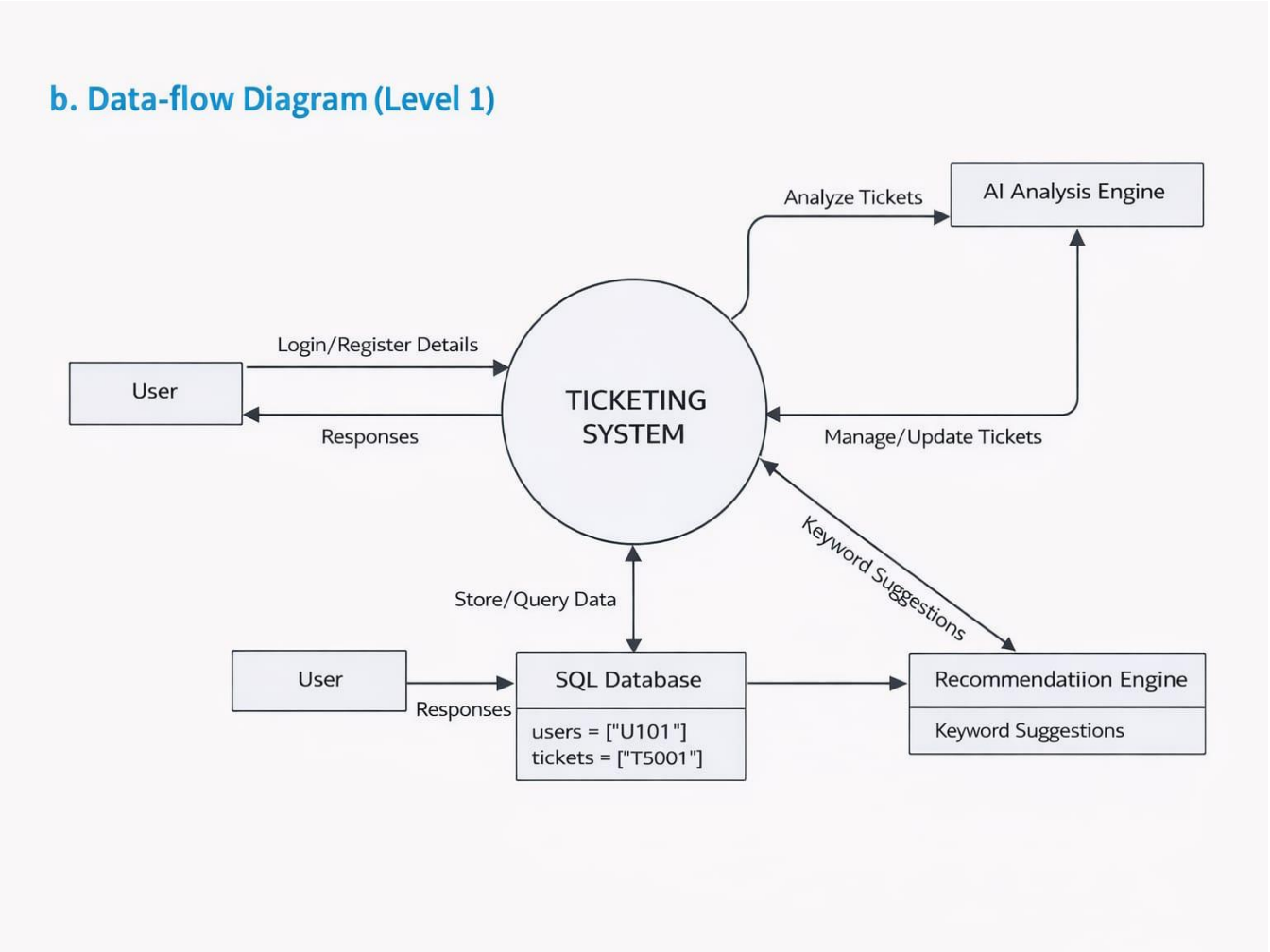


4.2 Data Design

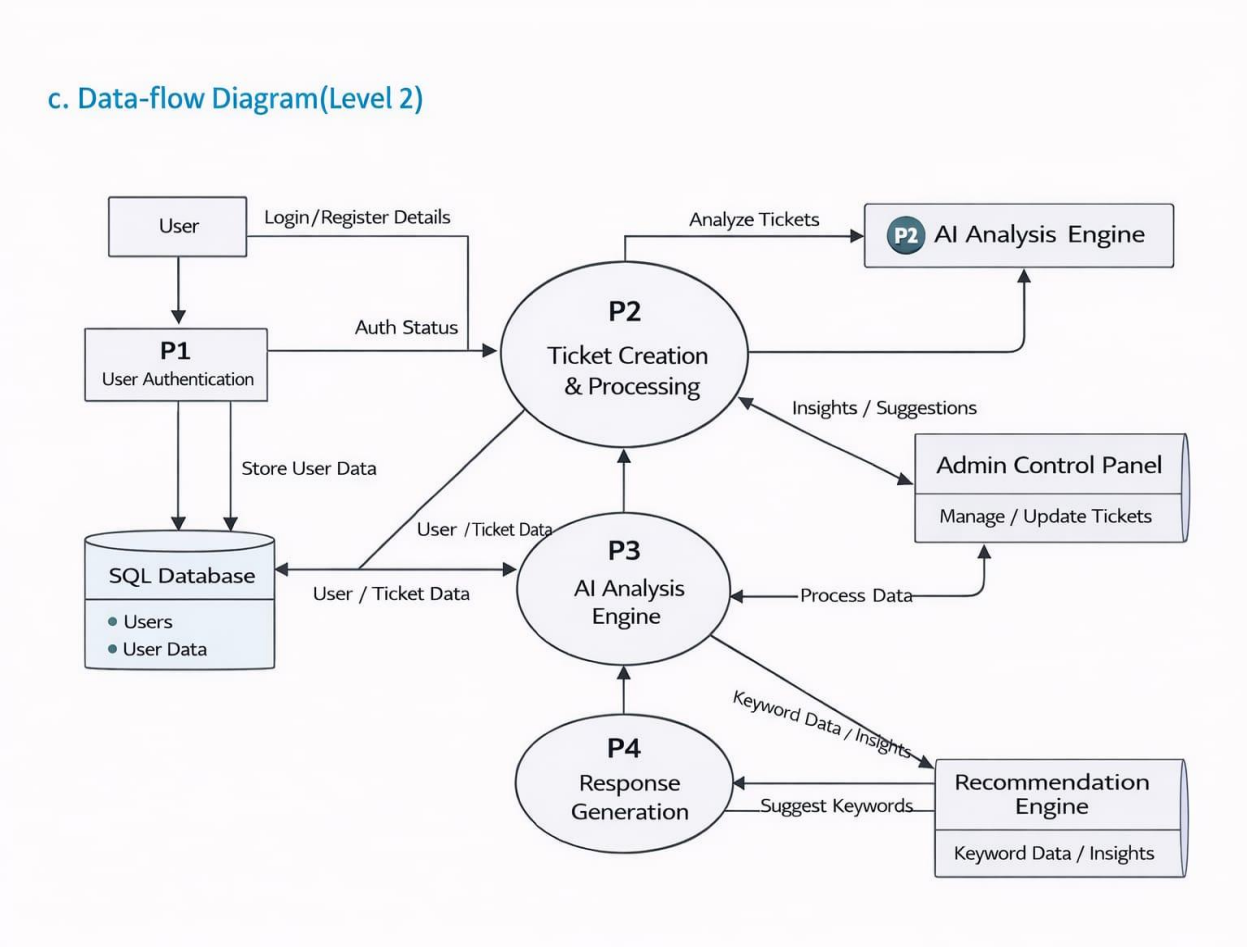
a. Data-Flow Diagram(Level 0)



b. Data-Flow Diagram(Level 1)

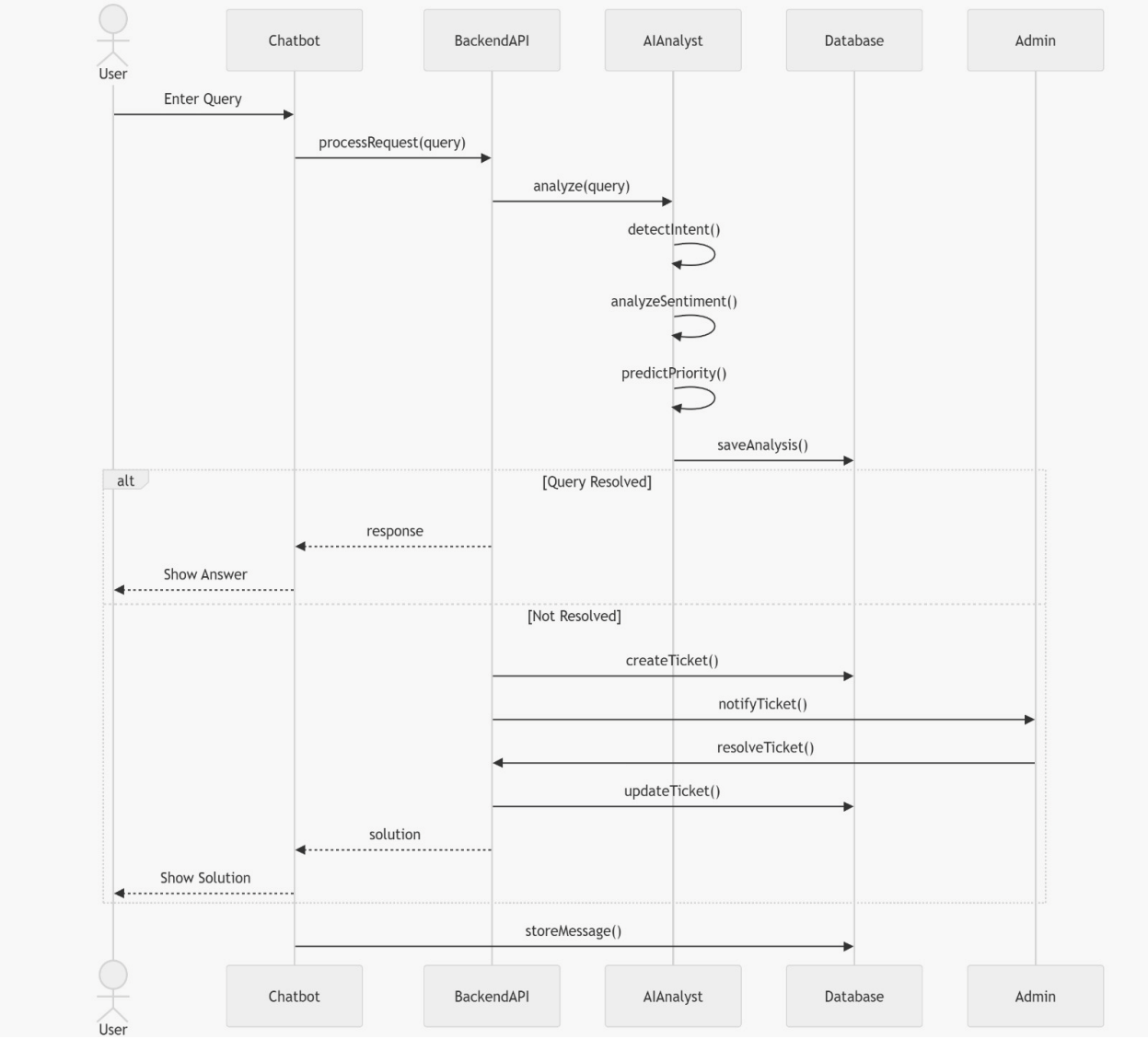


c. Data-Flow Diagram(Level 2)

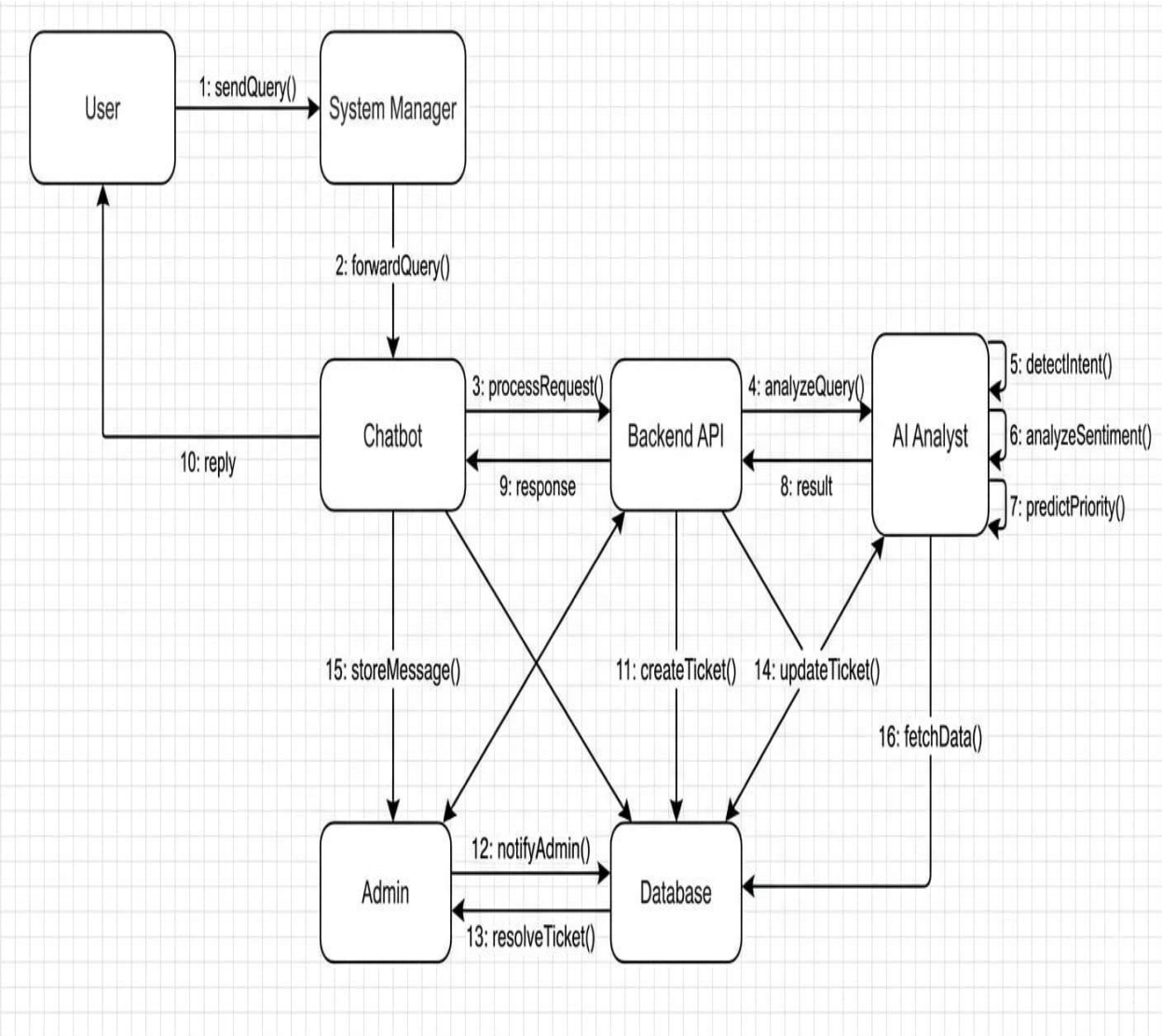


4.3. Behavioural Design

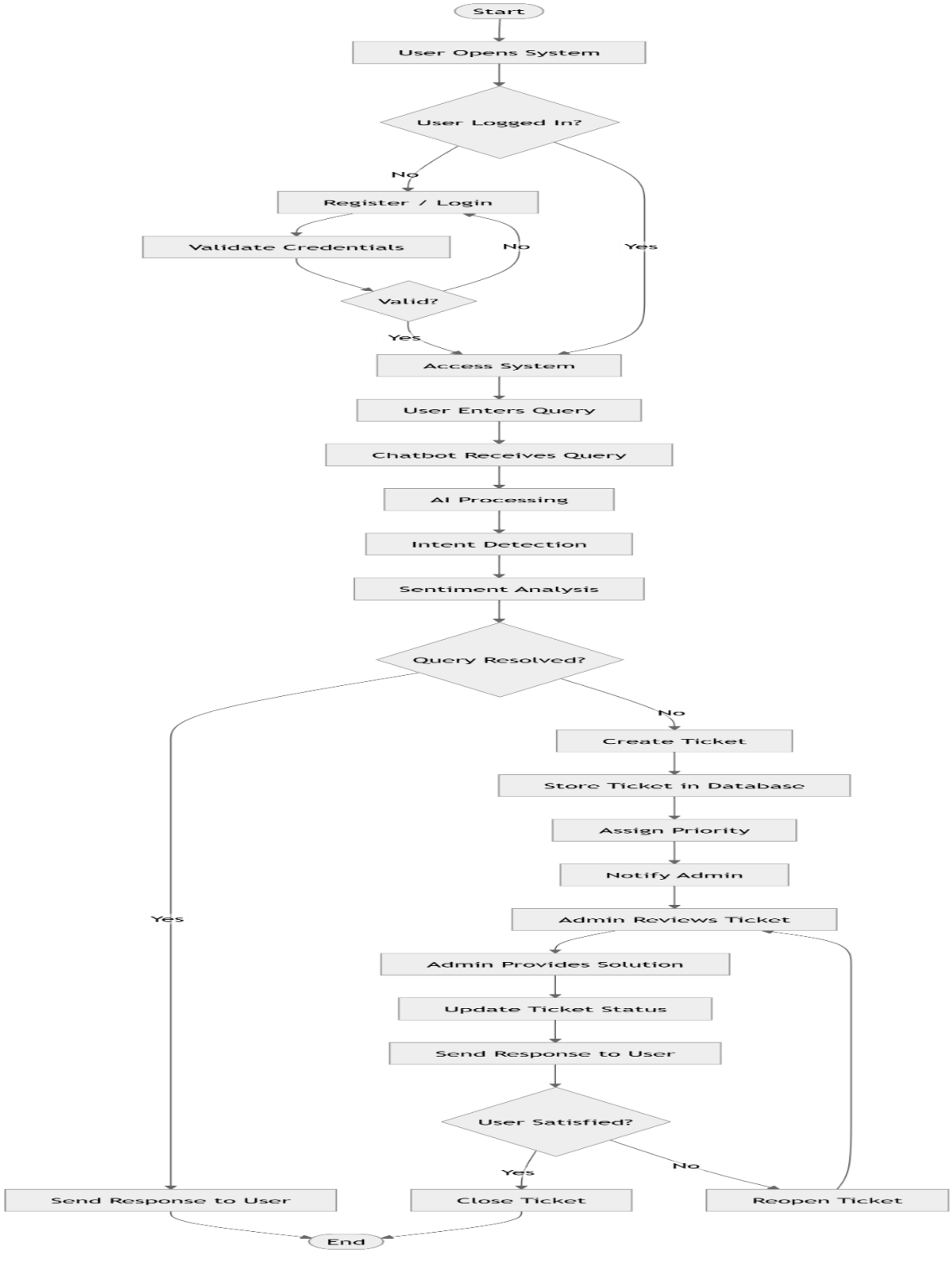
a. Sequence Diagram



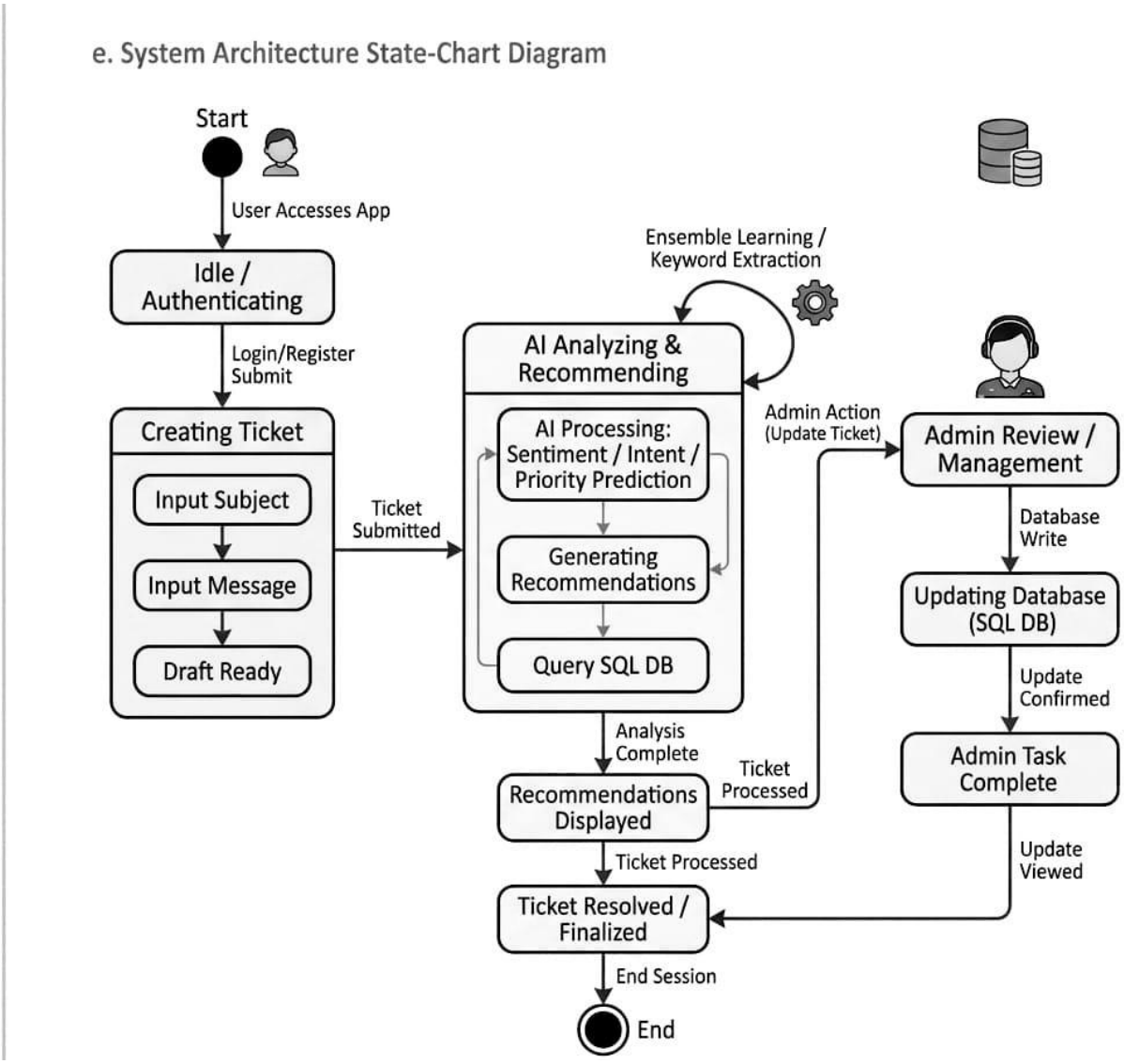
b. Collaboration diagram



c. Activity Diagram

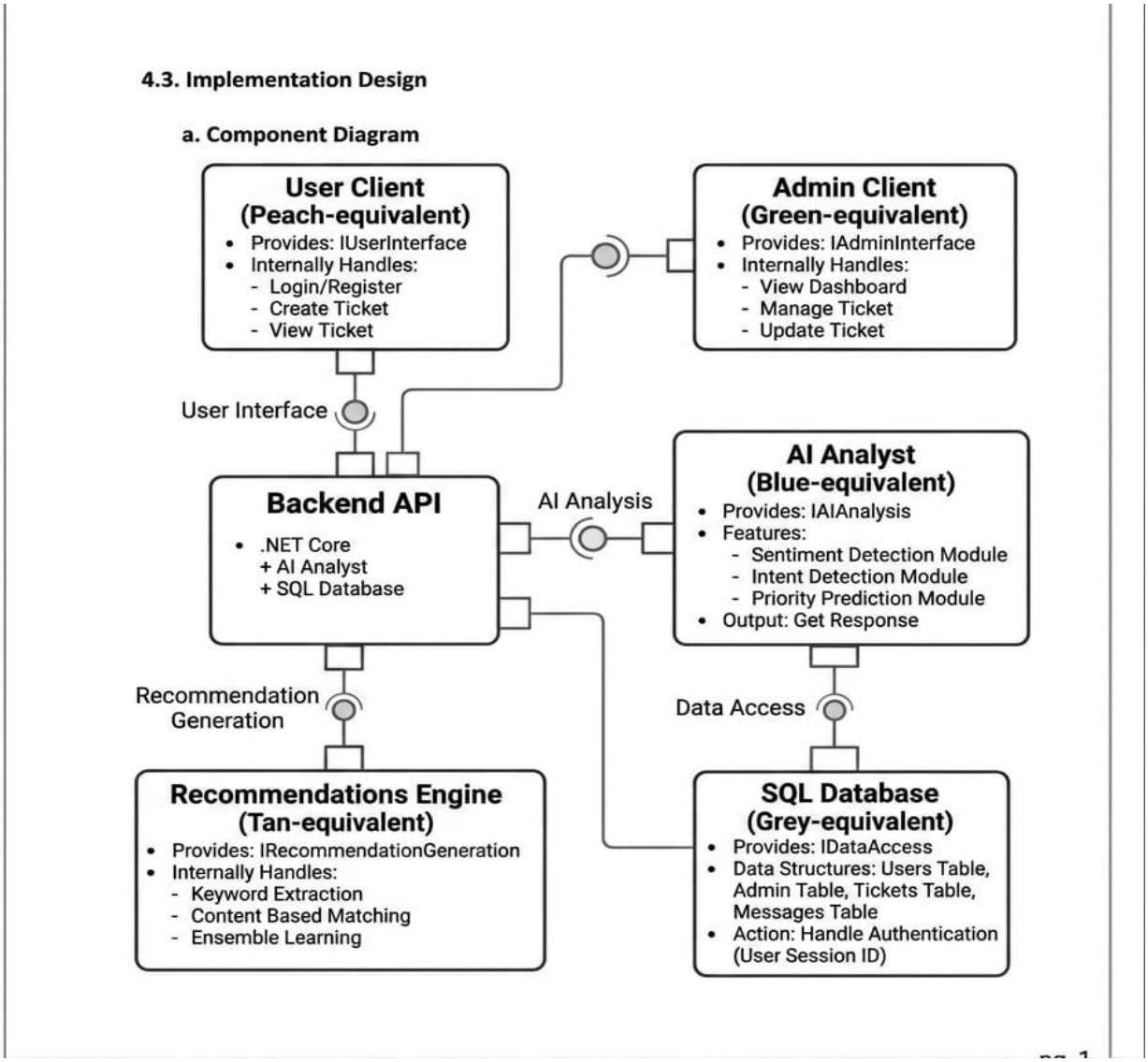


d. State-chart diagram



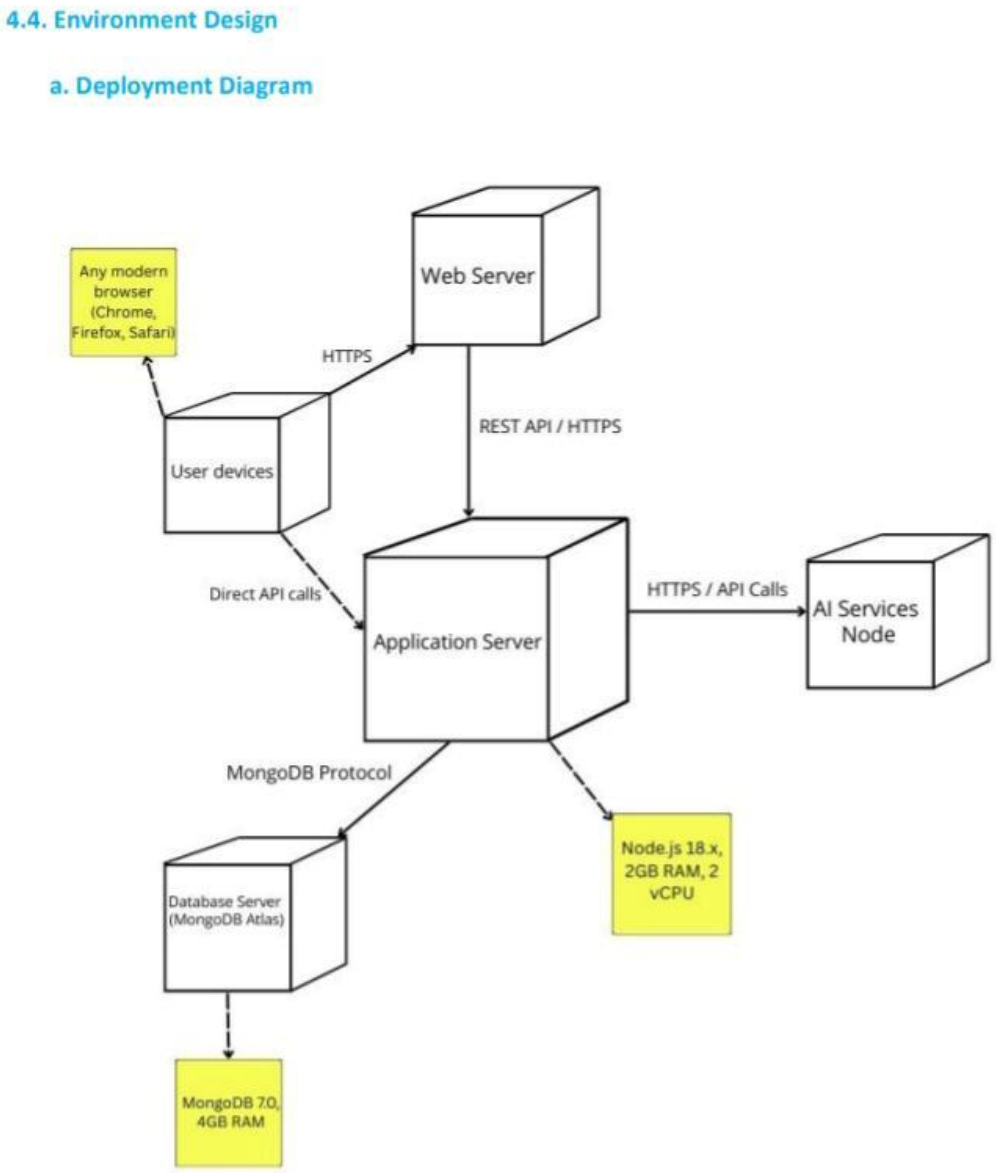
4.3. Implementation Design

a. Component Diagram



4.4. Environment Design

a. Deployment Diagram



5. Algorithms for Functional Requirements

5.1 User Authentication Algorithm (JWT Authentication)

Purpose:

Used to securely authenticate users and allow only authorized users to access the system.

Algorithm: JSON Web Token (JWT)

Working:

1. User enters email and password.
2. System validates credentials from SQL Server.
3. If valid, JWT token is generated.
4. Token is sent to frontend.
5. User includes token in future requests.
6. Backend verifies token before processing.

Input:

- Email
- Password

Output:

- Authentication Token

5.2 Chatbot Response Algorithm

Purpose:

Generate automatic responses for common customer issues.

Algorithm: Rule-Based Keyword Matching

Working:

1. User enters a message.
2. System extracts keywords.
3. Message is compared with predefined responses.
4. Matching reply is returned.
5. If no match → ticket is created.

Input:

- Customer message

Output:

- Chatbot response

5.3 Sentiment Analysis Algorithm

Purpose:

Detect customer emotion from messages.

Algorithm: TF-IDF + Naive Bayes

Working:

1. User message is converted into numerical vectors using TF-IDF.
2. Model analyzes text patterns.
3. Sentiment is predicted.

Input:

- Customer text

Output:

- Positive / Negative / Neutral

5.4 Ticket Classification Algorithm

Purpose:

Predict support ticket category.

Algorithm: Multinomial Naive Bayes

Working:

1. User issue is received.
2. Text preprocessing is performed.
3. ML model predicts category.

Categories:

- Technical
- Billing
- Account
- General

Output:

- Predicted Category

5.5 Ticket Priority Prediction Algorithm

Purpose:

Assign urgency level.

Algorithm: Rule-Based Priority Mapping

Working:

1. Sentiment is checked.
2. Negative → High Priority.
3. Neutral → Medium Priority.
4. Positive → Low Priority.

Output:

- High / Medium / Low

5.6 Database Storage Algorithm

Purpose:

Store structured and unstructured data.

Algorithm: Hybrid Database Strategy

Working:

- SQL Server → Authentication & Users
- MongoDB → Tickets & Chat History

Output:

- Persistent Data Storage

References:

1) AI-based Customer Support Systems

Authors: Zhang, Y., Chen, X.
Title: *Intelligent Customer Support Systems Using NLP and Machine Learning*
Journal: IEEE Access
Year: 2023

Summary:
This paper explains how NLP and machine learning can automate customer support by analyzing user queries and generating responses.

Use in your project:

- Chatbot design
- NLP-based query understanding

2. Sentiment Analysis in Customer Feedback

Authors: Kumar, A., Singh, R.
Title: *Sentiment Analysis of Customer Reviews Using Machine Learning Techniques*
Conference: International Conference on AI & Data Science
Year: 2022

Summary:
Discusses using Naive Bayes and Logistic Regression for detecting customer emotions from text.

Use in your project:

- Sentiment detection (Positive/Negative/Neutral)
- Priority assignment

3.Ticket Classification using ML

Authors: Li, H., Wang, T.
Title: *Automated Support Ticket Classification Using Text Mining*
Journal: Springer (Machine Learning Applications)
Year: 2024

Summary:
Focuses on classifying support tickets into categories using TF-IDF and classification algorithms.

Use in your project:

- Ticket category prediction
- AIService replacement with ML

4.Chatbot Systems for Customer Service

Authors: Adamopoulou, E., Moussiades, L.
Title: *Chatbots: History, Technology, and Applications*
Journal: Applied Sciences
Year: 2022

Summary:
Explains chatbot architectures including rule-based and AI-based approaches.

Use in your project:

- Hybrid chatbot design
- Rule-based + AI approach

5.Hybrid Database Systems (SQL + NoSQL)

Authors: Stonebraker, M., et al.
Title: *SQL vs NoSQL Databases: A Hybrid Approach*
Journal: Communications of the ACM
Year: 2023

Summary:
Discusses combining relational and NoSQL databases for scalable applications.

Use in your project:

- SQL Server + MongoDB architecture